Report

Created by	Chiara Visca
Created time	@October 17, 2025 5:17 PM
Category	Subproject 2 - Backend
Last edited by	Timothy Rasmussen
Last updated time	@December 15, 2025 11:38 AM

CIT/P Portfolio Project - Subproject 2: The Backend

Course: Complex IT Systems – Practice

Group: [12]

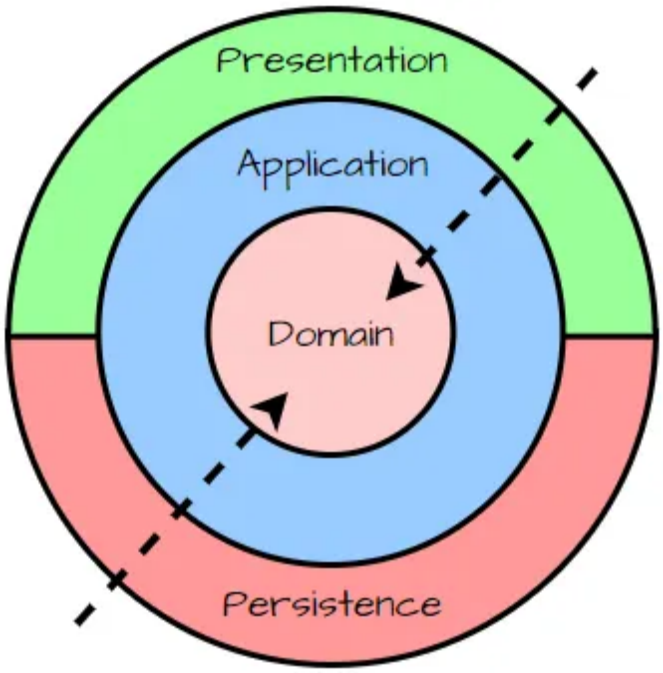
Authors: Christopher Bouet, Timothy Stoltzner Rasmussen, Mana Karki

1. Application Documentation

Architecture Overview

The system requires a REST backend that exposes CRUD features with our existing PostgreSQL database. A clear separation of concerns is needed to keep read-heavy IMDB data isolated from user-owned data and to ensure predictable paging and discoverability across endpoints.

We Adopted a 4 layered architecture, API (Presentation), Application (Service), Domain, Infrastructure (persistence).



The API layer provides versioned controllers with HATEOAS and pagination; the Application layer encapsulates use-cases and returns `Result<T>`; the Domain layer defines aggregates and contracts; the Infrastructure layer maps EF Core to the existing schema and provides repositories through Unit of Work.

- **API:** Versioned routes `/api/v{version}/...`, controllers for titles/persons, Swagger, and shared HATEOAS/paging utilities (`LinkDto`, `PagedResult<T>`, `UrlHelper.GeneratePaginatedUrl`, `ToPagedResult`) extentions for a cross system solution.

- **Application:** `TitleService`, `PersonService` expose queries/commands and return `Result<T>` of type `DTO` for explicit success/failure.
- **Domain:** Aggregates `Title`, `Person`, `Rating` with domain events; repositories defined via interfaces (e.g., `ITitleRepository`).
- **Infrastructure:** `MovieDbContext` maps to `movie_db` and `profile` without changing the schema; repositories and `UnitOfWork` implement persistence

The architecture keeps IMDB data read-only and profile data authoritative, enabling future expansion (e.g., additional controllers) without revisiting foundational choices. If a multi-database split becomes desirable, the repository/Unit of Work boundary and explicit mappings allow the context to be separated with minimal surface-area changes.

Key Design Decisions

1) Identifier Strategy (UUID + Legacy IMDB IDs)

- String IMDB IDs complicate type-safe APIs and cross-schema joins.
- Use UUIDs as primary keys; persist IMDB identifiers in `legacy_id` on `title` and `person`.
- All routes and services resolve by UUID; legacy lookups remain available for compatibility.
- Stronger referential integrity and EF compatibility, with preserved external linkage.
- The dual-identifier model supports analytics imports and external referencing without constraining internal typing.

2) HATEOAS + Pagination as Shared Infrastructure

- Inconsistent collection responses hinder client navigation and caching.

- Standardize on `PagedResult<T>` with `{self, first, prev, next, last}` links; default page size 20, maximum 100.
- `ToPagedResult` composes items, totals, and links from the current request context.
- Endpoints expose uniform pagination and discoverability, satisfying portfolio requirements 2-C.5/2-C.6.
- The same wrapper can be extended to remaining list endpoints (e.g., persons' auxiliary lists) for full coverage.

3) Repository + Unit of Work

- Direct EF usage in services couples business logic to persistence and weakens transaction control.
- Define repository interfaces per aggregate and coordinate them with a Unit of Work boundary.
- `IUnitOfWork` aggregates repositories and manages transactions for multi-entity workflows (e.g., ratings).
- Improves testability and ensures atomicity across tables.
- The boundary supports future cross-database orchestration without distributed transactions.

4) Result Pattern for Application Flow

- Exception-driven flow obscures expected outcomes and complicates HTTP mapping.
- Services return `Result<T>` with typed errors (e.g., `NotFound`, `Duplicate`).
- Controllers translate results to ProblemDetails with appropriate status codes.
- Error semantics are explicit and controller logic remains minimal.
- The pattern scales to additional endpoints without altering domain contracts.

5) EF Core with PostgreSQL Query Shaping

- Search-heavy endpoints require efficient projections and database-backed filtering.
- Use `AsNoTracking` and `Select` for read paths; employ PostgreSQL features (ILIKE, trigram/FTS) backed by existing indexes.
- Repositories return `(items, totalCount)` with stable ordering before `Skip/Take`.
- Queries remain performant and deterministic; tests acknowledge in-memory limitations and validate SQL behavior against Postgres.
- Additional read-models can be introduced without changing API contracts.

6) API Versioning

- The frontend requires a stable contract during iterative backend changes.
- Apply URL-segment versioning with `Microsoft.AspNetCore.Mvc.Versioning` and ApiExplorer.
- Routes follow `/api/v1/...`; Swagger groups by version; new breaking changes ship as new versions.
- Backward compatibility is maintained while enabling controlled evolution.
- The policy keeps migration costs predictable for the SPA as scope grows.

7) DTO/Entity Separation

- Exposing domain entities risks leaking invariants and internal evolution to clients.
- Publish dedicated DTOs (optional `Url` for self-links); keep entities internal to the domain.
- Controllers map domain results to DTOs; item self-links are set via route helpers.

- API contracts remain stable and HATEOAS-ready.
- DTOs allow additive fields (e.g., links, summaries) without affecting domain models.

8) Transactional Integrity for Multi-Entity Operations

- Account/title/rating flows span multiple aggregates and must be atomic.
- Enforce explicit transactions at the Unit of Work level.
- Services open a transaction boundary around coordinated writes, then commit or roll back.
- Operations preserve consistency without exposing transaction details to controllers.
- The approach is compatible with future outbox-based integration if multiple databases are introduced.

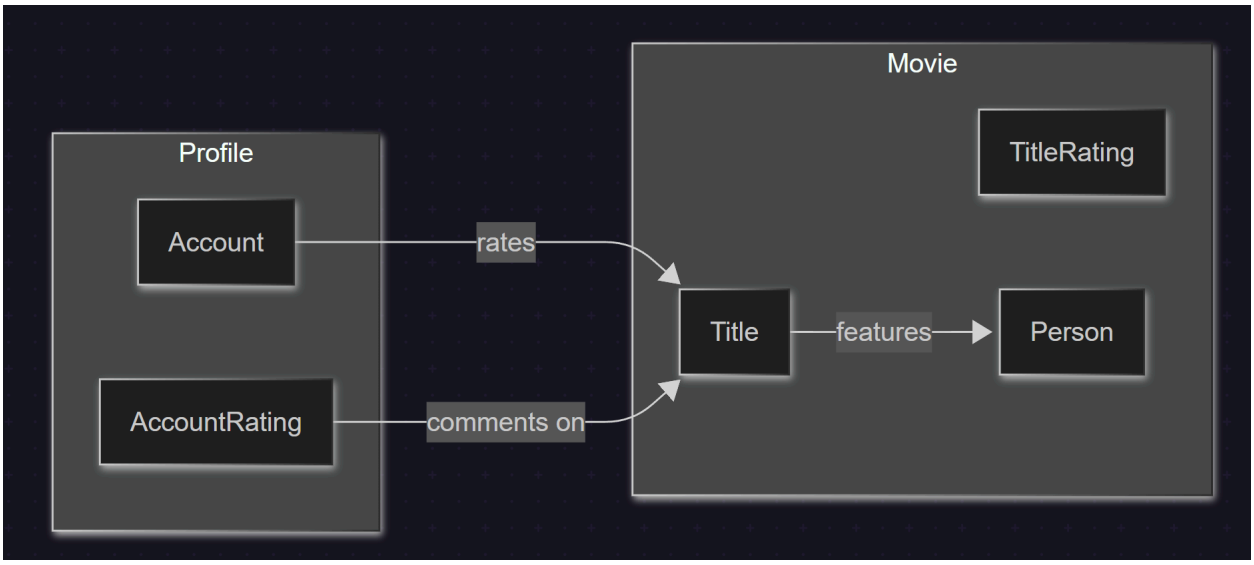
9) Database Reuse (No Migrations in This Subproject)

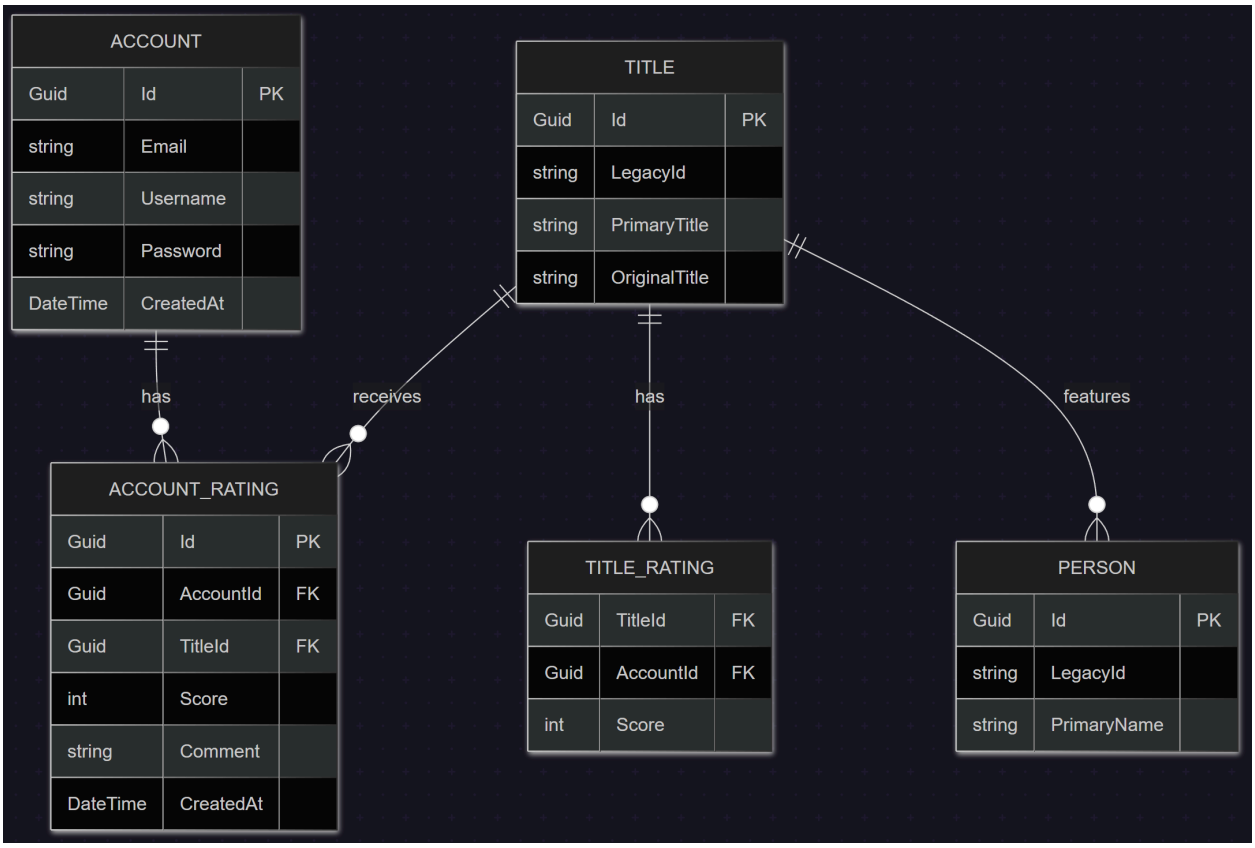
- Subproject 1 already provides a vetted schema and indexes.
- Map EF Core to existing schema objects; avoid schema mutations here.
- Schema-qualified mappings for `movie_db` and `profile`; materialized views and functions remain available for analytics.
- Responsibilities stay clean between database and backend; delivery risk is reduced.
- If domain needs change, migrations can be introduced in a future scope without disrupting current contracts.

2. Domain Boundaries

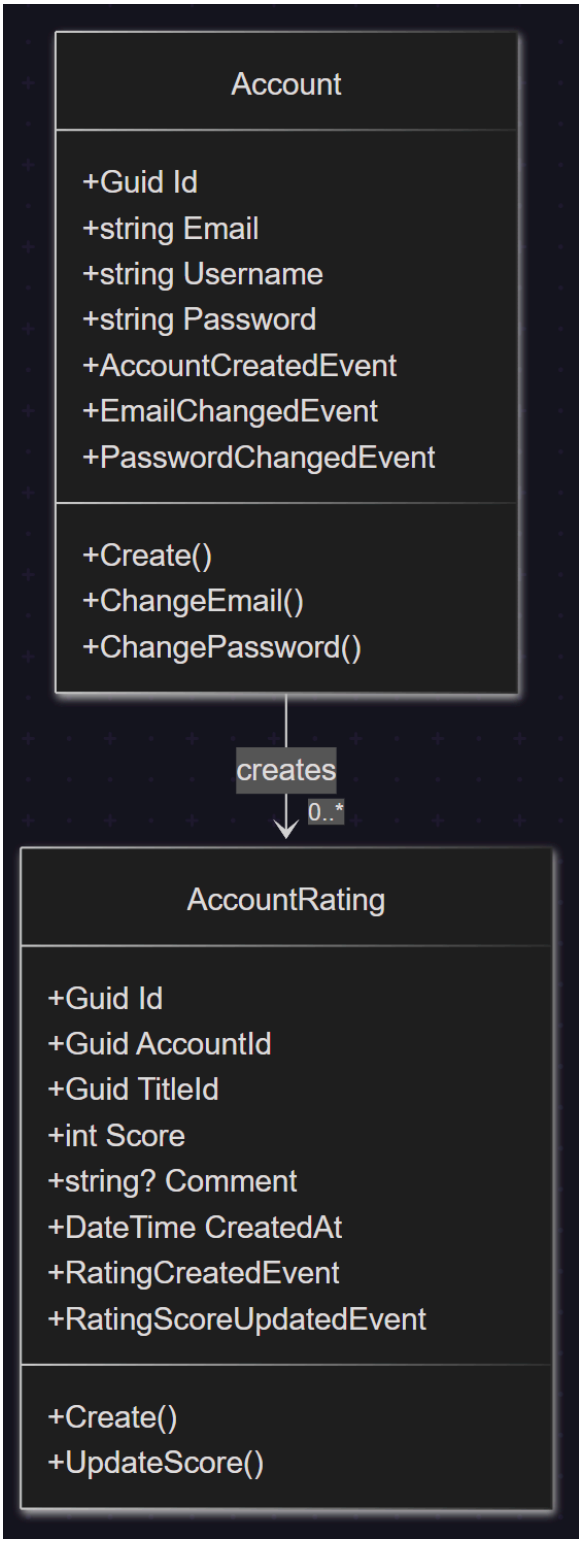
In the **domain boundaries**— Each bounded context defines its own aggregates (e.g., `Account`, `Title`, `Person`) and enforces local invariants. Interactions between

contexts occur through the application layer, ensuring that each model remains clean, explicit, and purpose-driven.




ACCOUNT
Guid
Id
PK
string
Email
string
Username
string
Password
DateTime
CreatedAt
TITLE
Guid
Id
PK
string
LegacyId
string
PrimaryTitle
string
OriginalTitle
has
receives
has
features
ACCOUNT_RATING
Guid
Id
PK
Guid
AccountId
FK
Guid
TitleId
FK
int
Score
string
Comment
DateTime
CreatedAt
TITLE_RATING
Guid
TitleId
FK
Guid
AccountId
FK
int
Score
PERSON
Guid
Id
PK
string
LegacyId
string
PrimaryName

Profile


Account
+Guid Id
+string Email
+string Username
+string Password
+AccountCreatedEvent
+EmailChangedEvent
+PasswordChangedEvent
+Create()
+ChangeEmail()
+ChangePassword()
creates
0..*
AccountRating
+Guid Id
+Guid AccountId
+Guid TitleId
+int Score
+string? Comment
+DateTime CreatedAt
+RatingCreatedEvent
+RatingScoreUpdatedEvent
+Create()
+UpdateScore()

Account

The `Account` aggregate encapsulates user identity, acting as the entry point for all profile-related interactions. It manages essential credentials including email, username, and password.

The entity is designed to enforce invariants through strict validation at creation and mutation points. It uses domain events (`AccountCreatedEvent`, `EmailChangedEvent`, etc.) to notify other bounded contexts or subsystems of critical state changes, adhering to eventual consistency principles. Interfaces (`IAccount`) support decoupling for dependency inversion.

- Constructor logic is internal or private to enforce controlled instantiation via the static `Create(...)` factory method.
- Email, username, and password are trimmed and validated to avoid state corruption.
- Mutations (email/password changes) check for identity before updating to prevent redundant events.

The class adheres to aggregate rules: changes are atomic, domain events are explicit, and side effects are not coupled to command handlers. Validation exceptions prevent illegal state. The entity avoids exposure of internal behavior and enforces encapsulation.

The domain could be extended with richer policies (e.g., email verification status, password complexity checks). Password hashing is assumed to occur elsewhere (e.g., in an application or infrastructure service), which should be documented.

Account Ratings

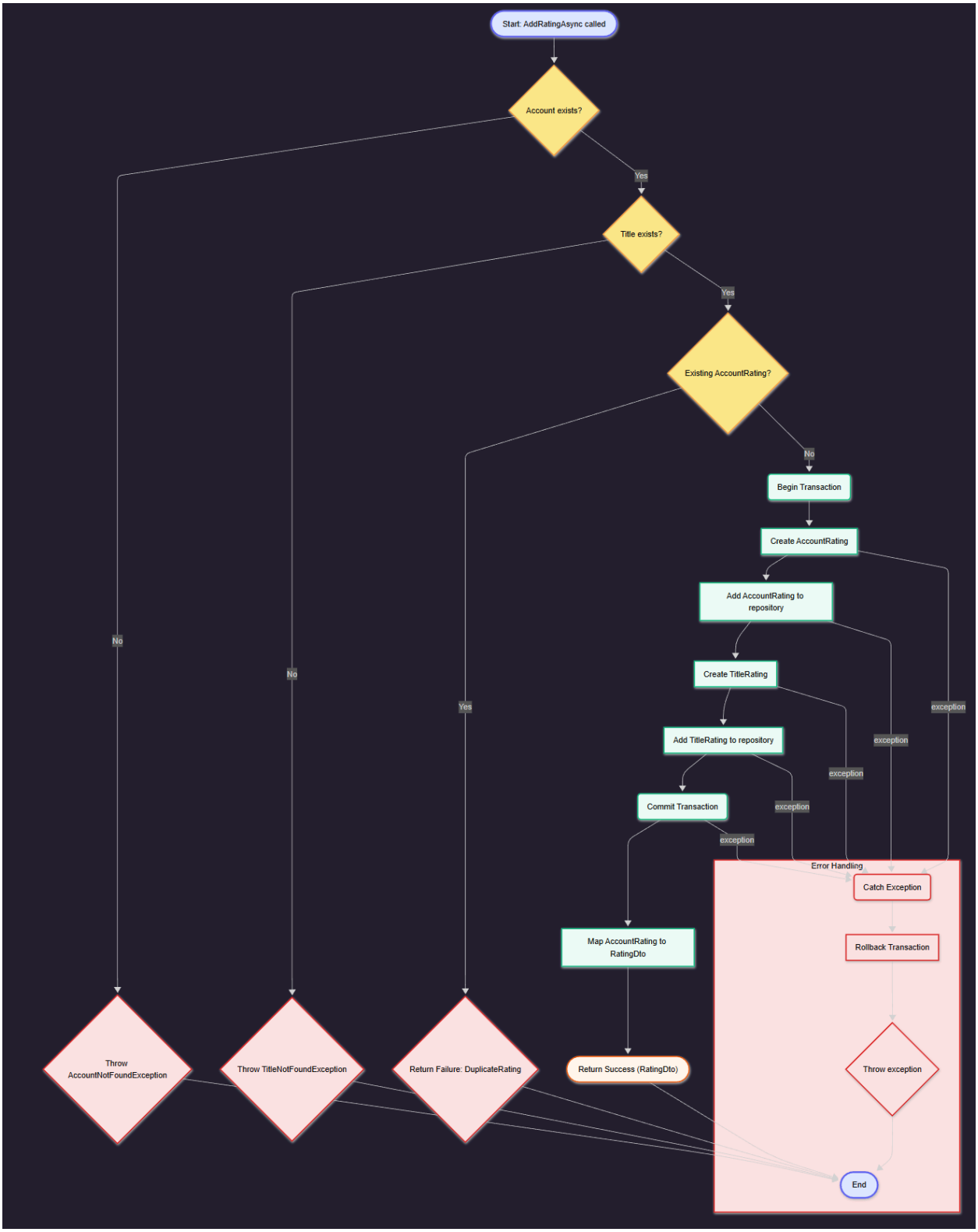
The `AccountRating` aggregate models an account's qualitative feedback on a specific title, with optional commentary.

Designed for expressive user interaction, it captures both quantitative (score) and qualitative (comment) input. Domain events (`RatingCreatedEvent`, `RatingScoreUpdatedEvent`) represent the lifecycle of the rating. It enforces business rules: score must be within [1–10], and comments (if present) must be concise (≤ 500 chars).

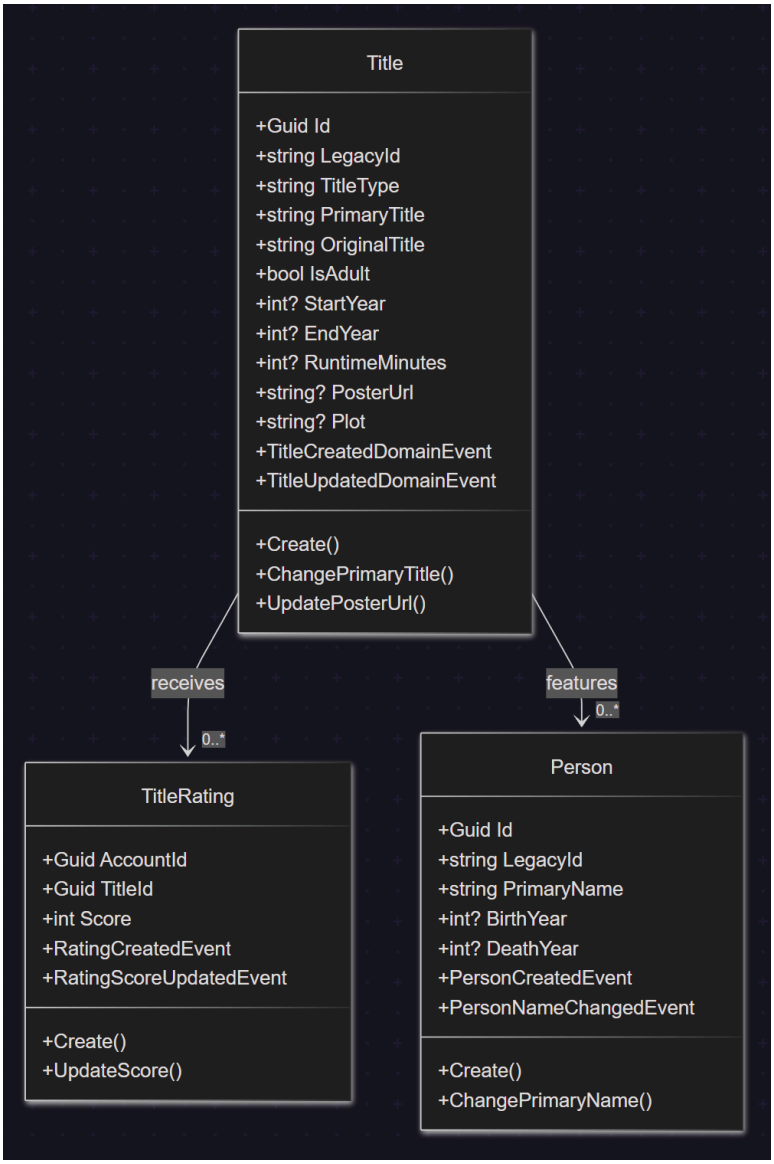
- Ratings are instantiated via the `Create(...)` method to centralize validation.
- Entity ensures immutability of keys (AccountId, TitleId) post-construction.
- Comment and score updates raise events and use UTC timestamps for consistency.

The design allows the system to respond reactively to rating changes (e.g., updating aggregate movie score). The model keeps temporal context via `CreatedAt`.

Future enhancements may include versioning for edits, audit trails, or sentiment analysis integration.


Start: AddRatingAsync called
Account exists?
Yes
No
Title exists?
Yes
No
Existing AccountRating?
Yes
No
Begin Transaction
Create AccountRating
Add AccountRating to repository
Create TitleRating
Add TitleRating to repository
Commit Transaction
exception
exception
exception
exception
exception
Error Handling
Catch Exception
Rollback Transaction
Throw exception
Map AccountRating to RatingDto
Return Success (RatingDto)
Throw AccountNotFoundException
Throw TitleNotFoundException
Return Failure: DuplicateRating
End

Movie



Title

The `Title` aggregate models a movie, series, or other media entry. It aggregates metadata such as identifiers, titles, release years, adult flag, runtime, poster, and plot.

Defaults are handled defensively: original title falls back to primary title, `IsAdult` defaults to `false`, and missing years are defaulted safely. Domain events (`TitleCreatedDomainEvent`, `TitleUpdatedDomainEvent`) track lifecycle changes. A static `GenerateLegacyId()` method ensures legacy compatibility.

- Factory method enforces the creation contract.
- Multiple update methods (for runtime, titles, poster, plot, etc.) provide fine-grained mutability.
- Redundant changes are skipped via value comparison.

The entity adheres to DDD principles: internal state is private, updates are intentional, and events reflect significant state changes.

Dynamic title updates may warrant event versioning. Consideration for localization or multi-language support could be factored in future domain expansions.

Title Ratings

The `TitleRating` aggregate captures a user's numerical evaluation of a title, typically rendered as a score between 1 and 10.

This model intentionally avoids commentary, distinguishing itself from `AccountRating`. It tightly binds `AccountId` and `TitleId` to a specific score, with all updates triggering domain events.

- The `Create(...)` method encapsulates input validation and event emission.
- Mutating the score uses an `UpdateScore(...)` method with event dispatch.
- Duplicate updates are avoided by early exits.

The model is purpose-built for performance and simplicity in scoring aggregation use cases. It is lightweight and sufficient for statistical computation.

Could be extended to support weighted ratings or timestamped rating history. Event consumers may aggregate or normalize data asynchronously.

Person

The `Person` aggregate represents a real-world individual associated with a title (e.g., actor, director). Key attributes include legacy identifiers, names, and

birth/death years.

Creation and mutation logic follow invariants: identifiers and names must be valid and trimmed. Domain events communicate identity establishment and name updates.

- Static `Create(...)` method enforces required fields and raises a `PersonCreatedEvent`.
- The `ChangePrimaryName(...)` method ensures idempotency and avoids unnecessary updates.
- Domain-specific exceptions (`InvalidLegacyIdException`, `InvalidPrimaryNameException`) clarify intent.

The class preserves historical data while allowing non-destructive updates. Validation at boundaries prevents invalid records from entering the domain.

Additional roles or aliases may be introduced via value objects or child entities to enrich the representation.

3. Data Access Layer Implementation

Data Access Overview

The backend reads IMDB-sourced data and manages user-owned records with low latency and strict ownership boundaries. Search endpoints are read-heavy and optimized around fast projections, while write paths guarantee transactional integrity across user-focused tables.

A single EF Core `MovieDbContext` is used, mapped to PostgreSQL schemas `movie_db` and `profile`. Schema-qualified mappings and existing `legacy_id` columns are preserved. Reads are optimized using lightweight projections and `AsNoTracking`, and search queries are delegated to PostgreSQL features—functions, views, and indexes—whenever they yield better performance or translation than LINQ.

Entities are mapped 1:1 to existing tables. No migrations are introduced. All paginated queries enforce deterministic ordering before applying `Skip/Take` and

return `(items, totalCount)` for API pagination. Search operations rely on `EF.Functions.ILike`, trigram or full-text indexes, and `api` schema functions where appropriate.

This setup delivers predictable paging, minimal tracking overhead, and efficient, index-driven filtering. Backwards compatibility is maintained via `legacy_id` lookups without sacrificing UUID primary keys.

If query complexity grows, dedicated read models, materialized views, or compiled EF Core queries can be introduced without changing API contracts or the underlying schema.

Repository Pattern

Using Entity Framework directly inside application services can easily lead to leaky abstractions, unclear transaction boundaries, and reduced testability. To avoid this, keep EF concerns isolated within the infrastructure layer and let application services depend only on well-defined repository and query interfaces.

Each aggregate should have a narrow repository interface, while read-specific needs should be handled by separate query interfaces. For example:

- **`ITitleRepository`** – `GetByIdAsync`, `GetByLegacyIdAsync`, and `SearchAsync(query, page, pageSize)` returning `(IEnumerable<Title> items, int totalCount)`.
- **`IPersonRepository`** – standard CRUD operations plus `ExistsByLegacyIdAsync`.
- **`IPersonQueriesRepository`** – read-only access with `AsNoTracking` projections, `ILike` filtering, ordered and paged results, and delegation to SQL functions for scenarios like co-actors, "known for" lists, or popularity rankings.
- **`IRatingRepository`** – per-account reads and concurrency-safe upserts with reliable lookup strategies.

With this separation, application services remain EF-agnostic and express use cases through commands and queries that return DTOs or domain entities where appropriate. This design makes unit testing easier using mocks or fakes, enforces cleaner boundaries, and keeps services focused solely on business logic and orchestration.

When new endpoints or use cases emerge, additional repositories or query handlers can be added without modifying existing contracts, keeping the system more maintainable and open to extension.

Unit of Work

Multi-entity operations—such as rating flows that modify account, title, and rating data—must execute atomically to maintain consistency. This requires explicit transaction boundaries and lifecycle management.

To achieve this, transaction control is centralized in an `IUnitOfWork`, which groups repositories and exposes methods like `BeginTransactionAsync`, `CommitTransactionAsync`, `RollbackTransactionAsync`, and `SaveChangesAsync`. Each application service defines a single transactional scope per use case.

Application services access repositories through the `IUnitOfWork`, perform coordinated changes across entities, and commit once at the end of the workflow. If an exception occurs, the transaction is rolled back. Read operations typically skip transactions unless snapshot-level consistency is required.

This approach ensures clear commit points, unified error handling, and predictable resource usage while preventing partial writes across aggregates.

Additionally, this transaction boundary becomes a foundation for future needs such as cross-context coordination, and it naturally supports patterns like outbox/inbox when integrating with external systems.

Portability to Multi-Database Deployment

Future separation of `profile` and `movie_db` into independent databases should not impact application logic or API contracts. To ensure this, all persistence concerns remain hidden behind repository interfaces and EF mappings that use schema-qualified configurations.

The current single `MovieDbContext` can be replaced with multiple database contexts—for example, `ProfileDbContext` and `MovieDbContext`—without touching domain services or controllers. Only connection strings and dependency injection registrations need to change. Each repository is bound to its specific context, and any workflow that spans multiple databases is orchestrated at the application service layer, without relying on distributed transactions.

Where eventual consistency is acceptable, an outbox pattern can be introduced to coordinate cross-database operations through reliable messaging.

This design keeps refactoring contained to composition and wiring rather than domain logic, minimizing migration risk and avoiding downtime.

If strict consistency across databases becomes a future requirement, the existing Unit of Work boundary can evolve into a saga-style coordinator. This preserves application service interfaces while swapping only the orchestration strategy.

---

4. Web Service Layer

Implementation Overview

The service layer must expose stable, discoverable endpoints for titles and persons, while shielding use-case logic from transport and persistence layers. Error behavior must be explicit and consistently translated to HTTP responses.

This is achieved through thin controllers (`PersonsController`, `TitlesController`) that delegate all behavior to application services. Controllers handle only request/response translation. All IO uses DTOs; domain entities stay internal to the application. Application services return `Result<T>` objects with strongly typed errors, which controllers translate into HTTP responses using `ProblemDetails`.

Cross-cutting conventions are applied uniformly:

- **API Versioning** via URL segments (`/api/v{version}/[controller]`)
- **HATEOAS** links for discoverability
- **Consistent pagination structures**
- **OpenAPI/Swagger** for contract visibility and documentation

DTOs can optionally include a `Url` field to expose their own self-link. Shared abstractions like `LinkDto`, `PagedResult<T>`, and response helpers ensure consistency across controllers.

This design keeps controllers small, testable, and free from business or persistence logic. Error handling remains uniform across endpoints, and the resulting API is self-describing and easy to navigate.

New resources can be introduced simply by adding new services and DTOs—without modifying existing endpoints, controller logic, or error policies—allowing the API surface to scale cleanly over time.

API Contract

Clients need predictable identifiers, media types, and response formats to efficiently support search, retrieval, and navigation at scale.

To achieve this, the API uses UUIDs as primary identifiers in resource URLs, while still exposing IMDB identifiers as `legacyId` for alternate lookup scenarios. JSON is the sole media type for all requests and responses, and collections follow a consistent wrapper shape that includes data, pagination metadata, and navigation links.

Routing conventions:

- `/api/v1/titles/{id}` – primary lookup by UUID
- `/api/v1/titles/{legacyId}` – alternate lookup by IMDB ID
- `/api/v1/titles?query=...` – searching or filtering

  (Same pattern applies for persons.)

Media type:

- `application/json` for all endpoints.

Collection format:

- `PagedResult<T>` containing:

- `items`, `page`, `pageSize`, `totalItems`, `totalPages`, and `links`.

HTTP status codes:

- `200` – successful reads
- `201` – resource created, with `Location` header
- `204` – successful idempotent write with no body
- `400`, `404`, `409`, `500` – errors serialized as RFC 7807 `ProblemDetails`

This contract is explicit, cache-friendly, and versionable. Clients can traverse the API using links rather than relying on out-of-band knowledge.

Future changes are additive—new fields or new versions—without breaking existing clients. IMDB-based lookups remain supported without compromising type safety or identifier consistency.

HATEOAS

Clients benefit from first-class navigation links to avoid manually assembling URLs and to enable cursorless pagination. This reduces coupling to route structures and improves interoperability across clients.

Each item DTO includes an optional `Url` (self-link), populated using route helpers in controllers. Collections implement canonical pagination links—`self`, `first`, `prev`, `next`, and `last`—generated centrally to avoid duplication.

These links are emitted through a `PagedResult<T>`, which is built using a `ToPagedResult(...)` helper that derives link values from the current request path and query parameters.

As a result, responses become self-descriptive: clients can follow links to related resources or additional pages without reconstructing URLs or relying on external documentation.

The same linking strategy extends cleanly to secondary collections such as “known-for” titles or co-actors, enforcing consistent navigation semantics across

the entire API surface.

Pagination

Search endpoints are high-volume and must guarantee consistent paging and stable ordering to ensure both correctness and a good user experience.

Paging Rules

- `page` must be ≥ 1
- Default `pageSize = 20`
- Maximum `pageSize = 100`
- Repository methods return `(items, totalCount)`
- Controllers wrap results in `PagedResult<T>` and add HATEOAS navigation links

Query Behavior

- Apply deterministic ordering before `Skip` / `Take`
  - e.g., sort by normalized title or person name
- Clients supply paging via `page` and `pageSize` query parameters
- Response shape always includes:
  - `items`
  - `page`, `pageSize`
  - `totalItems`, `totalPages`
  - HATEOAS pagination links (`self`, `first`, `prev`, `next`, `last`)

This guarantees that clients receive stable, predictable slices with accurate item counts. The approach is efficient, cache-friendly, and easy to reason about.

If demand grows or keyset-based performance is needed, cursor-based pagination can be introduced alongside page/size parameters without breaking existing consumers.

API Versioning

The frontend requires backward compatibility as endpoints evolve. To support this, the API uses URL-segment versioning with `Microsoft.AspNetCore.Mvc.Versioning`, integrated with `ApiExplorer` so each version is exposed in OpenAPI.

Routes follow the pattern `/api/v1/...`, and controllers or individual actions are annotated with their supported versions. When a breaking change is required, a new version such as `v2` is introduced, while older versions remain accessible during a deprecation period.

This makes version boundaries explicit and allows the SPA to upgrade specific endpoints independently rather than rewriting the entire API surface at once.

The approach enables parallel development of new capabilities while maintaining stability for existing consumers.

5 Testing

The system spans domain logic, Unit Of Work persistence, and HTTP endpoints, so testing must isolate responsibilities while still verifying that layers interact correctly.

To achieve this, the test strategy mirrors the architecture:

- **Domain and application tests** focus purely on logic, without any I/O or EF Core dependencies.
- **Infrastructure tests** run against a real PostgreSQL instance to validate repositories, EF Core mappings, and transactional behavior.
- **API and controller tests** exercise HTTP endpoints directly, including scripted flows, and assert status codes, DTO shapes, pagination metadata, and RFC 7807 `ProblemDetails`.

Test projects follow the structure of the production code. Shared fixtures supply deterministic data and object builders, enabling consistent setup across tests.

Assertions emphasize both behavior and contract accuracy.

Because each test targets a clearly defined boundary, failures are easy to trace to a specific layer, reducing diagnosis time and avoiding false positives caused by excessive mocking or incorrect fakes.

The test suite can be extended over time with SPA contract tests and performance baselines for high-traffic endpoints such as search.

---

6 Reflections and Discussion

Several architectural and design considerations emerged that were not explicitly addressed in previous sections. One of them concerns future support for multiple databases. Although the current solution operates on a single `DbContext`, the use of repositories, dependency injection, and schema-qualified EF Core mappings ensures that the application layer remains unaffected if the user-owned `profile` schema and IMDB-derived `movie_db` schema are later separated into distinct databases. Should workflows span across both, the outbox pattern would be a suitable mechanism for ensuring reliable messaging and eventual consistency.

HATEOAS has only been partially implemented. Search endpoints already expose navigation links and pagination metadata, whereas several list endpoints, particularly those related to persons, accounts, and ratings, still return raw collections. This creates an inconsistent client experience. Extending the existing link and pagination components to all list endpoints would provide a uniform contract and improve overall discoverability.

Another important design decision is the treatment of IMDB-derived data as strictly read-only. The system deliberately avoids exposing modification routes for these entities. All user-generated content, including rating history and account information, is instead stored exclusively in the `profile` schema. If additional metadata or annotations become necessary in the future, they should be modelled as auxiliary tables rather than altering the imported IMDB structures.

Related to this is the distinction between transactional and analytical data. Individual user interactions, such as ratings, are persisted in `profile.rating_history`, while aggregated rating data in `movie_db.rating` serves statistical and reporting purposes. This separation preserves data provenance and supports auditability, while still

enabling analytical queries. If real-time aggregates become necessary, asynchronous projections could be introduced without reworking the core write model.

From a collaboration perspective, the use of feature branches, small and reviewable pull requests, and clearly owned segments of the domain helped reduce coupling and refactoring conflicts. DTO conventions and explicit route versioning acted as the shared interface across teams. One improvement would be the introduction of a lightweight API changelog to complement Swagger and make version transitions more predictable.

Several lessons emerged during development. Test instability often stemmed from database-specific assertions being used in general unit tests; these were moved to isolated integration tests. Pagination initially produced inconsistent results due to missing ordering prior to `Skip` and `Take` operations; enforcing explicit ordering and centralizing URL generation resolved these issues.

Looking ahead, three areas require focus: completing HATEOAS and pagination across all endpoints, implementing authentication and authorization using JWT with a secure-by-default model, and improving operational performance for search-intensive interactions through compiled queries, caching static resources, and ongoing monitoring of query behaviour. These steps would strengthen the system's robustness, security, and consistency as it evolves.

8. Conclusion

Subproject 2 successfully delivers a maintainable, versioned REST backend that cleanly separates responsibilities, aligns with domain-driven design, and reuses Subproject 1's PostgreSQL schemas without schema mutations.

The backend now provides stable, evolvable API contracts with HATEOAS navigation, consistent pagination, EF Core mapping to existing schemas, clear transaction boundaries through repositories and Unit of Work, and a strong Result/DTO-based application flow.

This architecture enables the backend to scale functionally without introducing hidden coupling between layers or leaking persistence logic into business workflows. It also ensures that future teams can extend the system—either by

adding new endpoints, bounded contexts, or database sources—without refactoring foundational components.

The system is in a mature architectural state and ready for future extensions like authentication, full HATEOAS coverage, and multi-database separation.

The implementation meets the core portfolio requirements by providing:

- **HATEOAS-compliant endpoints (2-C.5)** using self-links and paginated navigational links.
- **Consistent, reusable pagination infrastructure (2-C.6)** shared across controllers.
- **Stable versioned contracts (`/api/v1/...`)** that isolate frontend-breaking changes.
- **Explicit application flows using `Result<T>`** for predictable HTTP translation and error handling.
- **Efficient search and projections** using EF Core with PostgreSQL features like ILIKE and trigram/FTS indexes.
- **Comprehensive testing at all relevant boundaries** (unit, integration, controller, and `.http`/Swagger validation).

Ready for Future Enhancements

- Completing HATEOAS + pagination coverage across all remaining list endpoints.
- Introducing authentication/authorization (JWT, roles, secure defaults).
- Scaling toward a multi-database setup by splitting `movie_db` and `profile` without impacting application logic.
- Optimizing performance further using compiled queries, caching, and read models.

Appendices

Appendix A: API Endpoint Reference

- Titles
  - POST `/api/v1/titles`
  - GET `/api/v1/titles/{id}`
  - GET `/api/v1/titles/by-legacy/{legacyId}`
  - GET `/api/v1/titles?query={q}&page={n}&pageSize={m}`
  - PUT `/api/v1/titles/{id}`
  - DELETE `/api/v1/titles/{id}`
- Persons
  - POST `/api/v1/persons`
  - GET `/api/v1/persons/{personId}`
  - GET `/api/v1/persons?query={q}&page={n}&pageSize={m}`
  - GET `/api/v1/persons/{name}/words?limit={k}`
  - GET `/api/v1/persons/{name}/co-actors`
  - GET `/api/v1/persons/{name}/popular-co-actors`
  - GET `/api/v1/persons/{personId}/known-for`
  - GET `/api/v1/persons/{personId}/professions`
- Accounts
  - POST `/api/v1/accounts`
  - GET `/api/v1/accounts/{accountId}`
- Ratings
  - POST `/api/v1/accounts/{accountId}/ratings`

- GET `/api/v1/accounts/{accountId}/ratings/{ratingId}`

Appendix B: Code Examples

- PagedResult wrapper (HATEOAS-ready)

```
public sealed class PagedResult<T>
{
    public IEnumerable<T> Items { get; init; } = Enumerable.Empty<T>();
    public int Page { get; init; }
    public int PageSize { get; init; }
    public int TotalItems { get; init; }
    public int TotalPages { get; init; }
    public Dictionary<string, LinkDto> Links { get; init; } = new();
}
```

- Controller paging usage

```
var paged = result.Value.items.ToPagedResult(
    result.Value.totalCount,
    page, pageSize,
    HttpContext,
    "SearchTitles",
    new { query }
);
return Ok(paged);
```

- URL helper for pagination links

```
public static string? GeneratePaginatedUrl(
    this IUrlHelper url, HttpContext ctx,
    string action, int page, int pageSize, object? extra = null)
{
    var baseUrl = $"{ctx.Request.Scheme}://{ctx.Request.Host}{ctx.Request.Pa
thBase}";
    var path = ctx.Request.Path.Value ?? "";
```

```
    var qs = new Dictionary<string,string?> { ["page"]=page.ToString(), ["page
Size"]=pageSize.ToString() };
    if (extra != null)
        foreach (var p in extra.GetType().GetProperties())
            qs[p.Name] = p.GetValue(extra)?.ToString();
    var q = string.Join("&", qs.Where(kv => kv.Value != null).Select(kv => $"{kv.K
ey}={Uri.EscapeDataString(kv.Value!)}"));
    return $"{baseUrl}{path}?{q}";
}
```

- DTO with self-link

```
public sealed record PersonDto(
    Guid Id, string LegacyId, string PrimaryName,
    int? BirthYear, int? DeathYear, string? Url = null);
```

Appendix C: Test Coverage Summary

- Unit tests
  - `Backend/cit12-portfolio-2/test-domain/PersonTests.cs`: entity validation and behavior (create, rename, invalid inputs).
  - `Backend/cit12-portfolio-2/test-application/TitleServiceTests.cs`: search returns `(items,totalCount)`, success/empty/error paths.
- Controller/API tests
  - `Backend/cit12-portfolio-2/test-api/TitlesControllerTests.cs`: `GetById`, `GetByLegacyId`, `Search` responses and ProblemDetails on errors.
- Integration tests
  - `Backend/cit12-portfolio-2/test-infrastructure/TitleRepositoryTests.cs`: EF Core mapping, search ordering, paging (notes: in-memory limitations for `ILike`).
  - `Backend/cit12-portfolio-2/test-infrastructure/UnitOfWorkPostgressTests.cs`: repositories wired with real context, transaction flow sanity.
- HTTP scripts (manual API checks)

- `Backend/cit12-portfolio-2/test-api/titles.http`: search pagination.
- `Backend/cit12-portfolio-2/test-api/accounts.http`: create/get account.
- `Backend/cit12-portfolio-2/test-api/test-all.http`: Create Account → Create Title → Create Rating end-to-end.
